



Universidad Nacional de Colombia

Matemáticas Discretas I

Taller 3 - Programación discreta

Criptografía, grafos, álgebra de Boole, Shannon y un primer vistazo cuántico

Docente:	Jhoan Sebastian Tenjo García
Modalidad:	Individual o en parejas. No se aceptan grupos de tres o más personas.
Entrega:	Repositorio en Git y documento breve de explicación en PDF.
Medio de entrega:	Enlace del repositorio por Classroom y PDF con la documentación.
Fecha de entrega:	La fecha exacta se indicará en Classroom.
Carácter:	Taller obligatorio del tercer corte.

1. Propósito

Este taller busca que las ideas finales del curso no se queden solo en definiciones. La idea es programar problemas pequeños, pero bien hechos, que muestren cómo aparecen las matemáticas discretas en seguridad, redes, circuitos lógicos, información y computación cuántica básica.

No se espera un software enorme. Se espera código claro, ordenado, verificable y explicado. Cada punto debe mostrar que el estudiante entiende qué está programando y por qué la solución funciona.

Idea central

El taller no consiste en copiar funciones de internet. Consiste en convertir una idea matemática en un programa que reciba datos, produzca una salida correcta y pueda ser revisado por otra persona.

2. Reglas generales de entrega

El taller debe entregarse en un repositorio de GitHub, GitLab o una plataforma equivalente. El repositorio puede ser público o privado. Si es privado, deben dar acceso al docente.

El repositorio debe tener, como mínimo, la siguiente estructura:

Archivo o carpeta	Qué debe contener
README.md	Descripción del taller, integrantes, lenguaje usado, instrucciones para ejecutar y lista de ejercicios desarrollados.
src/	Código fuente de los 10 ejercicios. Puede separarse por carpetas: <code>cripto/</code> , <code>grafos/</code> , <code>boole/</code> , <code>cuantica/</code> .
tests/	Pruebas o archivos de ejemplo que permitan verificar que el código funciona.
docs/	Documento en PDF o Markdown con la explicación matemática de las soluciones.
requirements.txt o similar	Librerías necesarias, si las usan. Si no usan librerías externas, indíquelo en el README.

Sobre el lenguaje de programación

Pueden usar Python, Java, C++, Julia, JavaScript u otro lenguaje, siempre que expliquen cómo ejecutar el proyecto. Python es recomendado porque permite escribir pruebas y visualizar resultados de manera sencilla.

3. Condiciones mínimas del código

Para cada uno de los 10 puntos deben entregar:

1. un archivo o módulo con la solución;
2. una forma clara de ejecutar el programa;
3. al menos tres pruebas o ejemplos de entrada y salida;
4. una explicación breve de la idea matemática usada;
5. comentarios suficientes para entender el código, sin llenarlo de texto innecesario.

No se aceptan soluciones que solo impriman la respuesta de un caso fijo. Cada programa debe funcionar para entradas nuevas del mismo tipo.

4. Uso de librerías

Se permite usar librerías, pero con criterio. Por ejemplo, pueden usar `networkx` para graficar o verificar resultados de grafos, pero si el punto pide implementar Dijkstra, deben programar la lógica principal. Pueden usar `sympy` para comparar una simplificación booleana, pero deben explicar qué está haciendo el programa.

Importante

Si usan una librería que resuelve completamente el problema sin que ustedes programen la idea principal, la solución no contará completa. El taller busca que ustedes implementen la matemática, no que llamen una función negra.

5. Problemas del taller

5.1. Bloque A. Criptografía

1. Cifrado César: romper un mensaje antiguo

Contexto. El cifrado César desplaza cada letra una cantidad fija de posiciones. Es sencillo, pero sirve para ver una idea clave de la criptografía clásica: ocultar un mensaje mediante una transformación reversible.

Qué deben programar. Un programa que permita:

- cifrar un texto usando un desplazamiento k ;
- descifrar un texto si se conoce k ;
- probar todos los posibles desplazamientos cuando k no se conoce.

Condiciones. Usen el alfabeto latino sin ñ para la versión principal: A, . . . , Z. El programa debe conservar espacios, signos de puntuación y números. Debe funcionar con mayúsculas y minúsculas, o convertir todo de manera consistente.

Ejemplo mínimo. Si el texto es HOLA UNAL y $k = 3$, el cifrado esperado es KROD XQDO.

Para la documentación. Expliquen por qué el descifrado usa el desplazamiento contrario y por qué el ataque de fuerza bruta es posible en este cifrado.

2. RSA de juguete: llaves, cifrado y descifrado

Contexto. RSA se basa en aritmética modular. En este taller se usará una versión pequeña, solo para entender la idea matemática. No se debe presentar como seguridad real.

Qué deben programar. Un programa que reciba dos primos p, q y un exponente público e , y que calcule:

- $n = pq$;
- $\varphi(n) = (p - 1)(q - 1)$;
- el inverso modular d de e módulo $\varphi(n)$;
- el cifrado $C \equiv M^e \pmod{n}$;
- el descifrado $M \equiv C^d \pmod{n}$.

Condiciones. Deben implementar el algoritmo de Euclides extendido o una función propia para calcular inversos modulares. El programa debe avisar si e no es válido, es decir, si $\gcd(e, \varphi(n)) \neq 1$.

Caso de prueba obligatorio. Con $p = 61$, $q = 53$, $e = 17$ y $M = 65$, deben obtener $n = 3233$, $\varphi(n) = 3120$, $d = 2753$, $C = 2790$ y al descifrar debe regresar $M = 65$.

Para la documentación. Expliquen el papel de los primos, del inverso modular y de la congruencia en el proceso.

3. MPC básico: calcular un promedio sin mostrar los datos

Contexto. En computación multipartita segura se quiere calcular algo entre varias personas sin revelar los datos individuales. En este punto no se construirá un protocolo real de seguridad industrial, sino una simulación clara de la idea.

Qué deben programar. Simulen un protocolo de suma secreta con tres servidores. Cada estudiante tiene una nota entera entre 0 y 50. Cada nota se divide en tres partes aleatorias módulo M . Cada servidor recibe una parte de cada nota. Al final, se reconstruye únicamente la suma total y el promedio.

Condiciones.

- Usen un módulo M suficientemente grande, por ejemplo $M = 1000003$.
- El programa debe permitir una lista de notas de cualquier tamaño.
- Ningún servidor por separado debe tener la lista original de notas.
- La salida principal debe ser la suma y el promedio, no los datos individuales.

Ejemplo mínimo. Si las notas son $[40, 35, 50, 25]$, la suma reconstruida debe ser 150 y el promedio 37,5.

Para la documentación. Expliquen con un ejemplo pequeño cómo una nota x se puede escribir como $x \equiv s_1 + s_2 + s_3 \pmod{M}$ sin que una sola parte revele x .

5.2. Bloque B. Grafos

4. Ruta más corta: una ciudad como grafo

Contexto. Una red de transporte puede representarse como un grafo ponderado: los vértices son puntos de la ciudad y las aristas son conexiones con un tiempo o distancia.

Qué deben programar. Implementen el algoritmo de Dijkstra para encontrar la ruta más corta entre dos vértices de un grafo ponderado con pesos positivos.

Condiciones.

- El grafo debe poder cargarse desde un archivo o desde una estructura clara en el código.
- El programa debe devolver la distancia total y la ruta encontrada.
- Deben incluir al menos un grafo de prueba con mínimo 8 vértices y 12 aristas.

Ejemplo de nombres. Pueden usar vértices como Portal, Calle26, Museo, Centro, Universidad, etc. No tienen que usar datos reales.

Para la documentación. Expliquen por qué Dijkstra necesita pesos no negativos y qué significa que un camino sea óptimo.

5. Cierre de una estación: medir el impacto en la red

Contexto. Si se cierra un punto importante de una red, algunas rutas se vuelven más largas y otras pueden desaparecer. Este punto busca medir ese cambio de manera sencilla.

Qué deben programar. Usen el grafo del punto anterior o uno nuevo. El programa debe:

- calcular rutas más cortas entre varios pares origen-destino antes del cierre;

- eliminar un vértice o una arista que represente el cierre;
- calcular de nuevo las rutas;
- reportar qué pares aumentaron su distancia y cuáles quedaron desconectados.

Salida esperada. Una tabla con columnas: origen, destino, distancia antes, distancia después, diferencia y estado.

Condiciones. Deben probar al menos cinco pares origen-destino. El programa debe identificar correctamente cuando no existe camino después del cierre.

Para la documentación. Expliquen cuál vértice o arista cerraron y por qué ese cierre produce o no produce un impacto importante.

6. Coloreo de grafos: organizar exámenes sin choques

Contexto. Si dos materias tienen estudiantes en común, no deberían tener examen al mismo tiempo. Esto se puede modelar con coloreo de grafos: cada color representa una franja horaria.

Qué deben programar. Un programa que reciba un grafo de conflictos y asigne colores a los vértices usando un algoritmo voraz.

Condiciones.

- El grafo debe tener mínimo 10 vértices.
- El programa debe verificar que no haya dos vértices adyacentes con el mismo color.
- Debe mostrar cuántos colores usó y qué vértices quedaron en cada color.

Ejemplo de interpretación. Los vértices pueden ser cursos y una arista significa que hay estudiantes inscritos en ambos cursos.

Para la documentación. Expliquen por qué el algoritmo voraz no siempre garantiza el menor número posible de colores, pero sí produce una asignación válida si se implementa bien.

5.3. Bloque C. Álgebra de Boole, Shannon y computación cuántica

7. Tablas de verdad y circuitos lógicos

Contexto. Las tablas de verdad permiten revisar todas las posibilidades de una expresión lógica. En electrónica digital, esas expresiones se interpretan como circuitos.

Qué deben programar. Un programa que genere la tabla de verdad de una expresión booleana con variables A, B, C, D y conectivos AND, OR, NOT y XOR. También debe permitir evaluar la expresión en una entrada concreta.

Condiciones. Pueden implementar un evaluador propio o definir las expresiones mediante funciones. El programa debe incluir mínimo tres expresiones, entre ellas:

$$(A \wedge B) \vee (\neg C), \quad (A \oplus B) \wedge C, \quad (A \vee B) \wedge (\neg A \vee C).$$

Salida esperada. Para cada expresión debe imprimirse una tabla clara con las combinaciones de entrada y el resultado final.

Para la documentación. Expliquen cómo se relaciona una tabla de verdad con un circuito lógico.

8. Simplificación booleana: hacer un circuito más barato

Contexto. Dos expresiones booleanas pueden tener la misma tabla de verdad, pero una puede necesitar menos compuertas. Simplificar no es solo un ejercicio algebraico: en circuitos reales significa ahorrar componentes.

Qué deben programar. Un programa que reciba una función booleana de tres o cuatro variables mediante sus minterminos y produzca una expresión simplificada en forma suma de productos.

Condiciones.

- Pueden implementar una simplificación básica por agrupación o una versión pequeña de Quine-McCluskey.
- Si usan una librería para comparar resultados, deben indicar cuál fue la salida de su propio programa y cuál fue la verificación.
- El programa debe comprobar que la expresión original y la simplificada tienen la misma tabla de verdad.

Caso de prueba sugerido. Para tres variables, prueben la función con minterminos $\{1, 3, 5, 7\}$. La simplificación esperada es una expresión equivalente a C , dependiendo del orden de variables usado.

Para la documentación. Expliquen qué significa un mintermino y por qué dos expresiones son equivalentes si tienen la misma tabla de verdad.

9. Shannon: medir información en un mensaje

Contexto. Claude Shannon propuso una forma de medir cuánta información contiene una fuente. Un mensaje muy repetitivo tiene menos incertidumbre que uno más variado.

Qué deben programar. Un programa que reciba un texto y calcule:

- frecuencia de cada símbolo;
- probabilidad de cada símbolo;

- entropía de Shannon

$$H = - \sum_i p_i \log_2(p_i);$$

- una comparación entre dos textos distintos.

Condiciones. Deben probar al menos dos mensajes: uno muy repetitivo y otro más variado. El programa debe explicar cuál tiene mayor entropía y por qué.

Extensión opcional. Implementar un código de Huffman básico y comparar la longitud promedio del código con la entropía.

Para la documentación. Expliquen en palabras sencillas por qué la entropía mide incertidumbre y no simplemente longitud del texto.

10. Primer simulador cuántico: bits, qubits y mediciones

Contexto. Un bit clásico vale 0 o 1. Un qubit se representa como una combinación $\alpha |0\rangle + \beta |1\rangle$, donde $|\alpha|^2$ y $|\beta|^2$ son probabilidades. En este punto no se usará un computador cuántico real: se hará una simulación pequeña con vectores.

Qué deben programar. Un simulador básico de un qubit que permita aplicar las compuertas X , Z y H :

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Condiciones.

- Representen el estado como un vector de dos entradas.
- Después de aplicar compuertas, calculen las probabilidades de medir 0 y 1.
- Simulen 1000 mediciones y reporten las frecuencias observadas.

Casos de prueba obligatorios.

- $X |0\rangle = |1\rangle$.
- $H |0\rangle$ produce probabilidades cercanas a 50 % y 50 %.
- $HH |0\rangle = |0\rangle$, salvo errores numéricos pequeños.

Para la documentación. Expliquen la diferencia entre probabilidad cuántica simulada y la ejecución en un computador cuántico real.

6. Documento de explicación

Además del código, deben entregar un documento breve en PDF. No debe ser largo, pero sí claro. Para cada punto respondan:

1. ¿Qué problema resuelve el programa?
2. ¿Qué idea matemática usa?

3. ¿Cómo se ejecuta?
4. ¿Qué pruebas hicieron?
5. ¿Qué limitaciones tiene la solución?

Sugerencia de estilo

Escriban como si otro estudiante del curso fuera a leer su repositorio para aprender. Eviten frases vacías. Muestren ejemplos concretos, capturas de salida o tablas pequeñas cuando ayuden.

7. Criterios de evaluación

La calificación del taller será sobre **5.0**. Se tendrá en cuenta tanto el funcionamiento del código como la explicación.

Criterio	Puntaje máximo
10 ejercicios implementados y funcionando	2.50
Correcta relación con los conceptos matemáticos del curso	0.80
Pruebas, ejemplos de entrada y salida, y verificación de casos límite	0.60
Repositorio ordenado, reproducible y con instrucciones claras	0.50
Documento de explicación completo, concreto y bien escrito	0.40
Claridad del código, nombres adecuados y comentarios útiles	0.20
Total	5.00

8. Recomendaciones para trabajar

- No intenten resolver todo el taller en una sola noche. Dividan el trabajo por bloques.
- Primero hagan una versión simple que funcione. Después mejoren el estilo y las pruebas.
- Hagan commits frecuentes. Un repositorio con un solo commit final no muestra bien el proceso.
- Lean sus propias instrucciones de ejecución desde cero. Si ustedes no pueden correr el proyecto en otro computador, probablemente el docente tampoco podrá hacerlo.
- No usen claves, datos personales ni información sensible real en los ejemplos.

9. Sobre el uso de herramientas de ayuda

Pueden consultar libros, apuntes, documentación oficial y ejemplos pequeños. También pueden usar herramientas de asistencia para resolver dudas puntuales. Sin embargo, el código entregado debe ser entendido por ustedes.

Si en la revisión se les pregunta por una función, una prueba o una decisión del algoritmo, deben poder explicarla. Si no pueden explicar lo que entregaron, esa parte no contará completa.

Cierre

Este taller es una invitación a ver el curso desde otra perspectiva. César, RSA, MPC, grafos, circuitos, entropía y qubits parecen temas separados, pero todos comparten la misma idea de fondo: trabajar con estructuras discretas, reglas precisas y procedimientos que se pueden ejecutar.



Jhoan Sebastian Tenjo García
Docente - Matemáticas Discretas I